

Java Practical: Abstract Classes and Interfaces

Instructions:

Complete the following exercises to reinforce your understanding of abstract classes and interfaces in Java. Write and test your code in a Java development environment.

Exercise 1: Implementing an Abstract Class

1. Create an abstract class `Animal` with the following:
 - An instance variable `name`.
 - A constructor to initialize `name`.
 - An abstract method `makeSound()`.
 - A concrete method `displayInfo()` that prints the animal's name.
2. Create two subclasses, `Dog` and `Cat`, that extend `Animal` and provide implementations for `makeSound()`.
3. In the `main` method, create objects of `Dog` and `Cat` and call their methods.

Exercise 2: Implementing an Interface

1. Define an interface `Vehicle` with the following methods:
 - `start()`
 - `stop()`
2. Create two classes, `Car` and `Bike`, that implement `Vehicle` and provide method implementations.
3. In the `main` method, create objects of `Car` and `Bike` and call their methods.

Exercise 3: Combining Abstract Classes and Interfaces

1. Create an abstract class `Appliance` with:
 - An instance variable `brand`.
 - A constructor to initialize `brand`.
 - An abstract method `showFunction()`.
 - A concrete method `displayBrand()` that prints the brand name.

2. Define an interface `SmartDevice` with a method `connectToWiFi()`.
3. Create a class `SmartFridge` that extends `Appliance` and implements `SmartDevice`.
Provide implementations for both methods.
4. In the `main` method, create an object of `SmartFridge` and call all available methods.

Exercise 4: Multiple Inheritance using Interfaces

1. Define an interface `Flyable` with a method `fly()`.
2. Define another interface `Swimmable` with a method `swim()`.
3. Create a class `Duck` that implements both `Flyable` and `Swimmable`.
4. Provide implementations for `fly()` and `swim()` methods in `Duck`.
5. In the `main` method, create an instance of `Duck` and call its methods.

Exercise 5: Smart Transport System

Design a smart transport system that includes different types of vehicles with self-driving capabilities. Follow these requirements:

1. Create an abstract class `Transport` with:
 - A `String model` attribute.
 - A constructor to initialize `model`.
 - An abstract method `move()`.
 - A concrete method `showModel()` to print the model name.
2. Define an interface `SelfDriving` with a method `autoPilot()`.
3. Create two classes `AutonomousCar` and `AutonomousDrone`:
 - Both extend `Transport` and implement `SelfDriving`.
 - Provide specific implementations for `move()` and `autoPilot()`.
4. In the `main` method:
 - Create objects of `AutonomousCar` and `AutonomousDrone`.
 - Call their respective methods.